

Microservices Data and Persistence

Give each service a persistence story: JPA and transactions, database-per-service, saga and outbox patterns, and the banking constraints around consistency and audit.



DAY 2 OBJECTIVES AND CAPSTONE PROGRESS

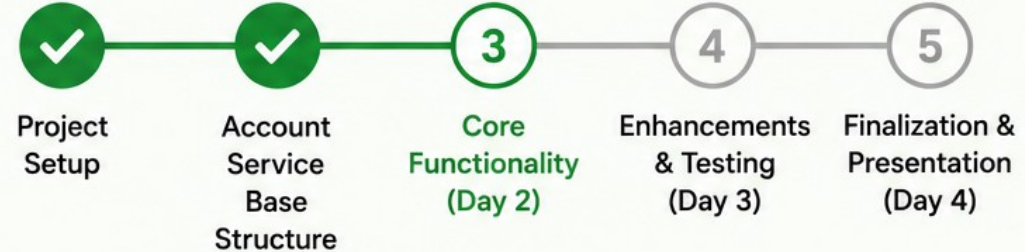


DAY 2 OBJECTIVES

-  Build and integrate Account microservice
-  Implement data persistence with Spring Data JPA
-  Complete REST APIs and test with Postman
-  Implement validation and exception handling
-  Write unit tests and ensure code quality



CAPSTONE PROGRESS



- | | | |
|---|--------------------|--|
|  | COMPLETED | Project setup, dependencies, and repository initialized |
|  | COMPLETED | Account service base structure created, layers and configurations in place |
|  | IN PROGRESS | Implementing CRUD operations, REST APIs, and data persistence |
|  | PENDING | Add validations, exception handling, and integration testing |
|  | PENDING | Finalize code, documentation, and capstone presentation |



CAPSTONE GOAL

Deliver a fully functional Account microservice with robust APIs, persistence, validation, testing, and high code quality.

DATA OWNERSHIP IN MICROSERVICES · DATABASE PER SERVICE PATTERN



BANKING OWNERSHIP EXAMPLE

- Account Service -> account records, status, nickname, customerId field
- Account Service -> account records, account status
- Transaction Service -> transaction ledger entries
- Risk Assessment Service -> risk scores, policy disposition, audit fields



BAD DESIGN: SHARED DATABASE

- Account Service, Transaction Service, Risk Assessment Service -> shared_bank_db
- Services look separate in code but are tightly coupled through data



WHY OWNERSHIP MATTERS

- Supports independent schema evolution and deployment
- Other services access data through REST APIs, events, or read models -- never direct queries



BETTER DESIGN: DATABASE PER SERVICE

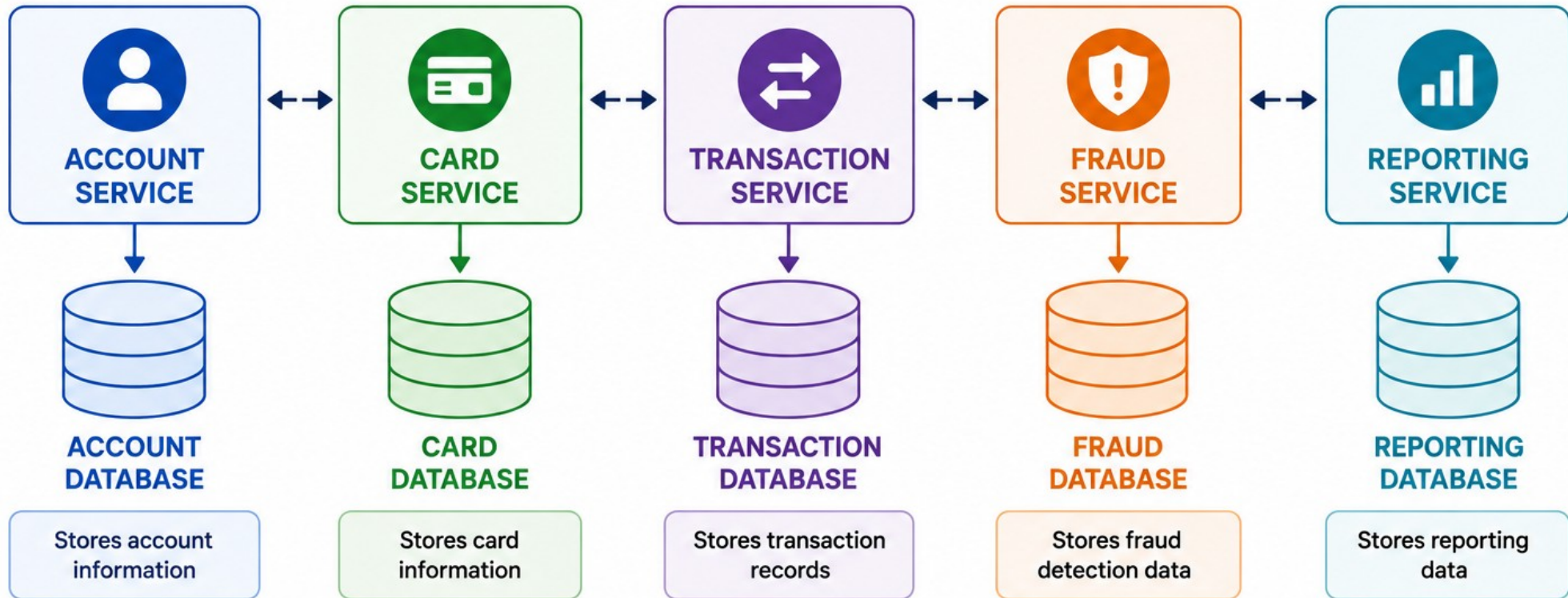
- Account Service -> account_db, Transaction Service -> transaction_db, Risk Assessment Service -> risk_db
- Services communicate through APIs and events rather than shared tables

KEY TAKEAWAY Account Service owns accounts, Transaction Service owns transactions, Risk Assessment owns scores -- no service reaches into another's tables. • Each service gets its own database or schema -- shared tables would silently recreate the coupling microservices exist to remove.

DATABASE PER SERVICE



Each microservice owns and manages its own database



BENEFITS



Loose coupling between services



Independent scalability



Technology flexibility



Better data security & privacy



Faster delivery and innovation

SHARED DATABASE ANTI-PATTERN

A schema change made for one service can silently break every other service reading the same shared tables.



WHY IT'S AN ANTI-PATTERN

- Multiple services become dependent on the same schema, tables, and release timing
- Cross-service queries and joins seem convenient but hide real coupling
- Designing service boundaries around database tables only leads to the same trap

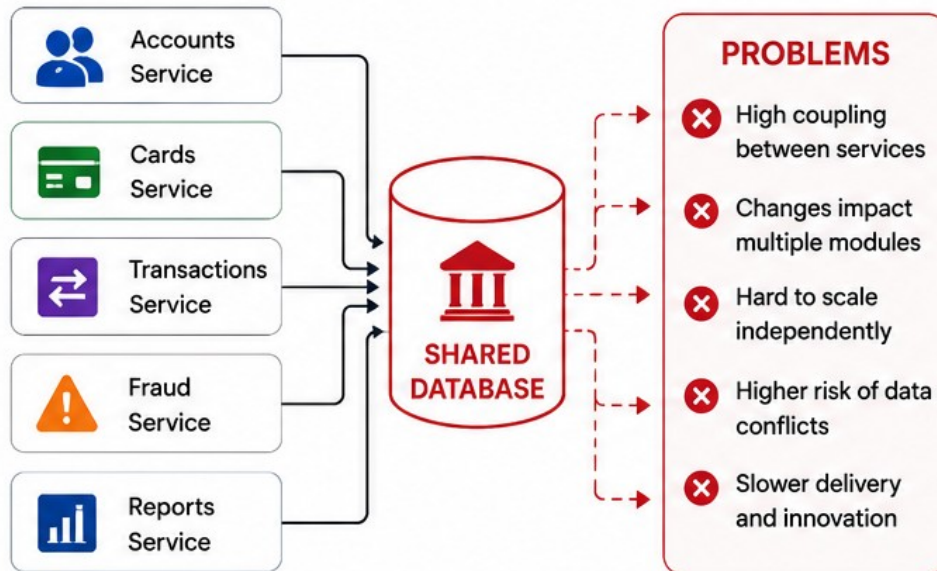
KEY TAKEAWAY A schema change made for one service can silently break every other service reading the same shared tables.

WHY SEPARATE DATA OWNERSHIP MATTERS IN BANKING

CENTRALIZED DATA OWNERSHIP



All data in a single database shared by all modules

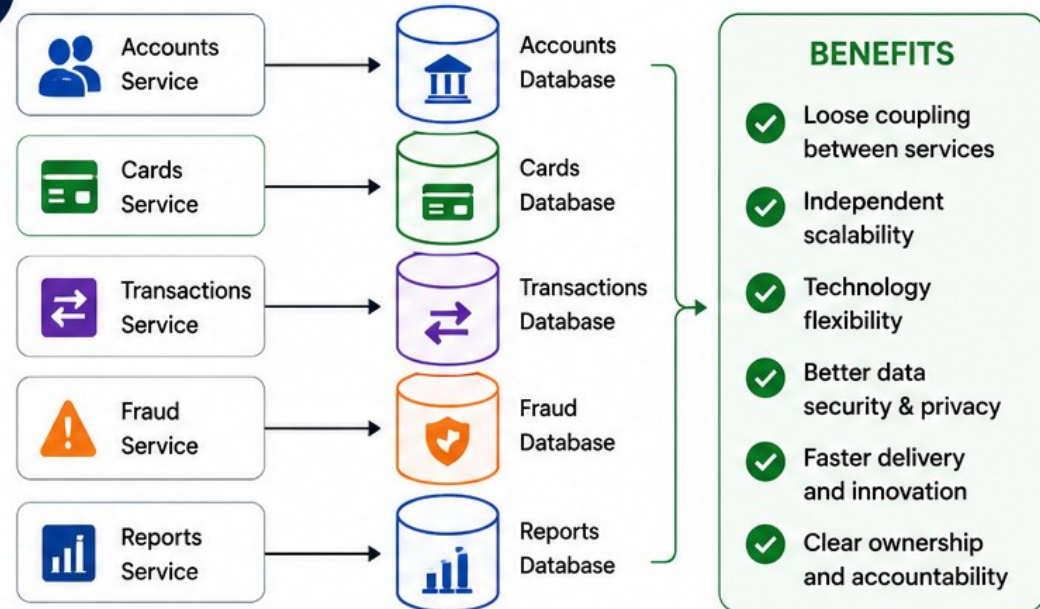


VS

SEPARATE DATA OWNERSHIP



Each service owns and manages its own data



**BUSINESS
IMPACT**



**Stronger Data
Security & Privacy**

Sensitive banking data
is isolated and protected.



**Better Scalability
& Performance**

Scale only what you
need, when you need.



**Faster Innovation
& Time to Market**

Teams move faster
without dependencies.



**Regulatory
Compliance**

Easier to meet data
residency and compliance
requirements.



**Lower Risk
& Cost**

Reduced impact
of failures and
lower operational cost.

POLYGLOT PERSISTENCE

No single database is best for every workload -- Order Service, Payment Service, and Analytics Service can each pick their own fit-for-purpose store.



EXAMPLE DATA STORE CHOICES

- Order Service -> PostgreSQL (relational, ACID, transactions, joins)
- **Lecture only:** some enterprises pick a document store for profiles -- this capstone uses PostgreSQL for every service
- **Lecture only:** Redis is a common cache -- Labs 1-5 do not install Redis
- Analytics Service -> Elasticsearch (full-text search, analytics)



CHALLENGES TO MANAGE

- Increased operational complexity and more skills/tools to manage
- Data consistency across different stores; no joins across databases (use APIs or events)

KEY TAKEAWAY No single database is best for every workload -- Order Service, Payment Service, and Analytics Service can each pick their own fit-for-purpose store.

Relational vs. NoSQL Data Stores



Choosing the right data store depends on your workload, data model, consistency needs, and scalability requirements.

 RELATIONAL DATA STORES (SQL)		 NoSQL DATA STORES (Non-Relational)
 Data Model	Structured data in tables (rows and columns) with a predefined schema.	Flexible data models: key-value, document, wide-column, or graph.
 Relationships	Strong support for relationships using foreign keys and joins.	Limited or no joins; relationships are typically handled within the application.
 ACID Transactions	Full ACID compliance ensures strong consistency and reliability.	Often BASE (Basically Available, Soft state, Eventually consistent) for higher availability.
 Scalability	Primarily vertical scaling (scale up). Horizontal scaling is more complex.	Designed for horizontal scaling (scale out) across many servers.
 Performance	Excellent for complex queries, transactions, and aggregations on structured data.	High performance for large volumes of simple reads/writes and unstructured or semi-structured data.
 Schema	Schema-on-Write: schema is defined upfront and enforced.	Schema-on-Read: schema is flexible and can evolve over time.
 Use Cases	Financial systems, ERP, order management, reporting, data with complex relationships.	Real-time analytics, user profiles, content management, IoT, catalogs, caching, event data.
 Examples	 PostgreSQL  MySQL  Microsoft SQL Server	 mongoDB  cassandra  Redis  elasticsearch



Key Takeaway: Relational databases provide strong consistency and complex query capabilities, while NoSQL databases offer flexibility and scale. Many microservices architectures use **polyglot persistence**—choosing the best data store per service.

SPRING DATA ARCHITECTURE · SPRING DATA JPA FUNDAMENTALS



THE SPRING DATA LAYERS

- Your Spring Boot application -> Spring Data abstraction layer -> store-specific module -> data store
- **Store modules:** Spring Data JPA, Spring Data MongoDB, Spring Data Cassandra, Spring Data Redis
- **Repository hierarchy:** Repository -> CrudRepository -> PagingAndSortingRepository -> JpaRepository



JPA FUNDAMENTALS

- **Entity:** represents a database table, mapped with @Entity
- **Primary key:** identifies each entity uniquely, mapped with @Id
- **Relationships:** @OneToMany, @ManyToOne, @OneToOne map associations
- **Persistence context:** a first-level cache that tracks entity state within a transaction

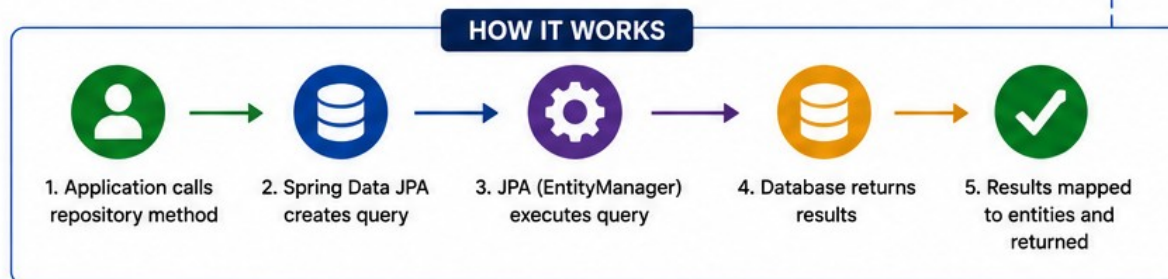
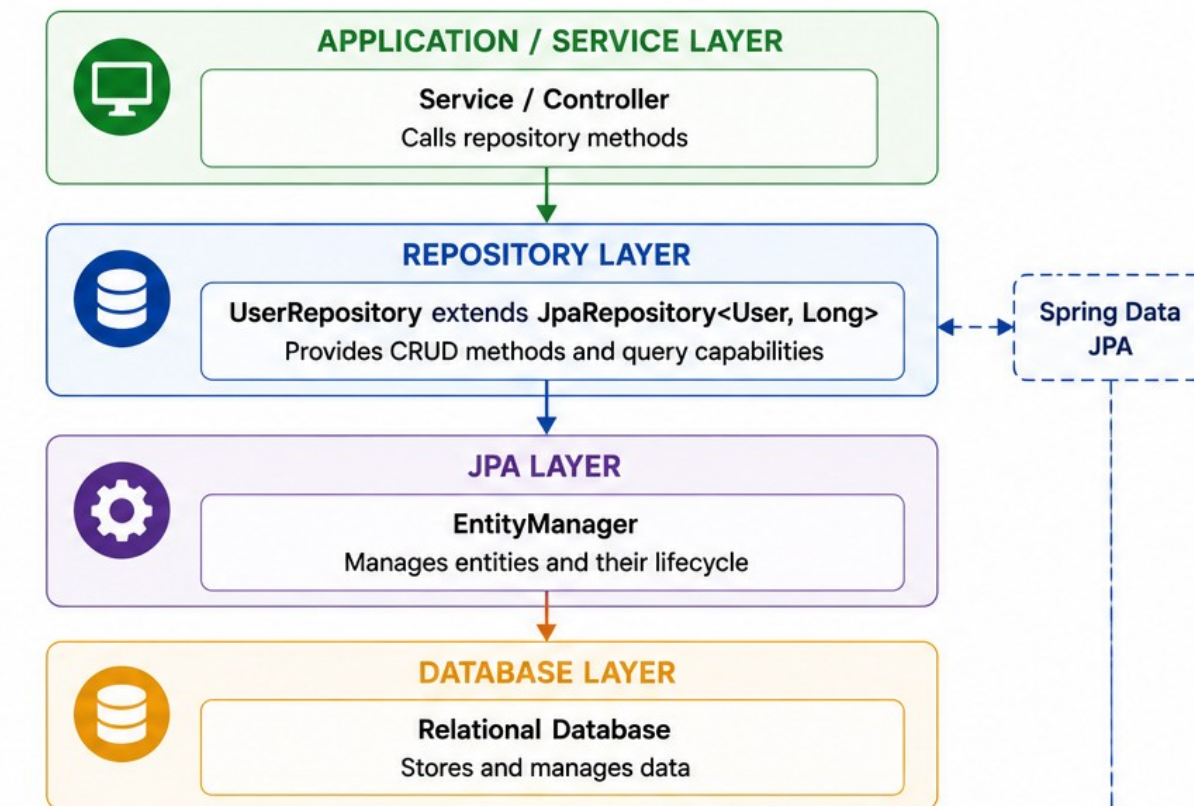


BEST PRACTICES

- Prefer derived queries for simple use cases; use @Transactional on service-layer methods
- Avoid exposing repositories outside the service layer

KEY TAKEAWAY Repository abstraction, template abstraction, domain mapping, and query derivation work the same way whether the store is JPA, MongoDB, or Redis. • Define the entity, extend JpaRepository, and Spring Data JPA generates the SQL -- no boilerplate DAO code required.

SPRING DATA JPA AND REPOSITORY PATTERN



BENEFITS



Reduces boilerplate code



Faster development



Powerful query capabilities



Type-safe and reliable



Seamless integration with Spring ecosystem

EXAMPLE

1. ENTITY

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;

    // getters and setters
}
```

2. REPOSITORY

```
public interface UserRepository
    extends JpaRepository<User, Long> {

    // Derived query method
    Optional<User> findByEmail(String email);

    // Custom query
    @Query("SELECT u FROM User u WHERE u.name = :name")
    List<User> findByName(@Param("name") String name);
}
```

3. SERVICE USAGE

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public User getUserById(Long id) {
        return userRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("User not found"));
    }
}
```

ENTITIES AND OBJECT-RELATIONAL MAPPING

The entity manager, persistence context, dirty checking, and SQL generation together turn changed Java objects into executed SQL statements.



HOW ORM WORKS

1. Entity manager manages the persistence context
2. Persistence context tracks entity states (managed, new, detached, removed)
3. Dirty checking detects changes in entities
4. ORM converts changes to SQL statements and executes them against the database



BEST PRACTICES

- Model the domain, not the database; keep entities small and focused
- Use appropriate fetch types (LAZY by default); keep business logic in the service layer, not the entity

KEY TAKEAWAY The entity manager, persistence context, dirty checking, and SQL generation together turn changed Java objects into executed SQL statements.

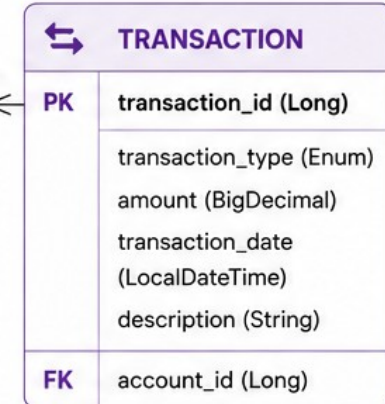
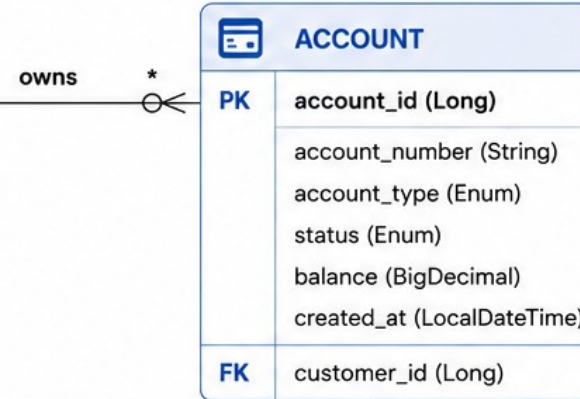
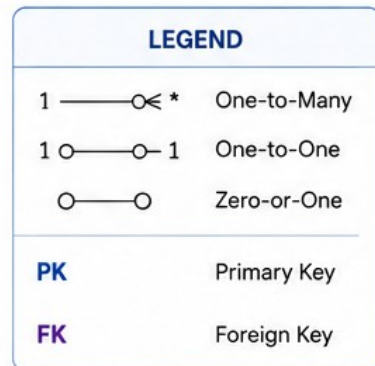
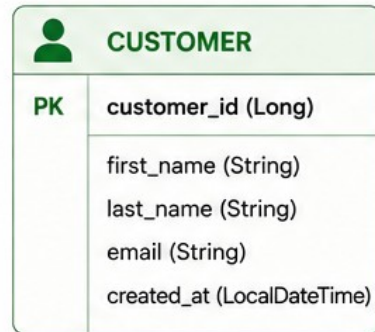
ENTITY DESIGN AND RELATIONSHIPS (WITHIN A SERVICE)



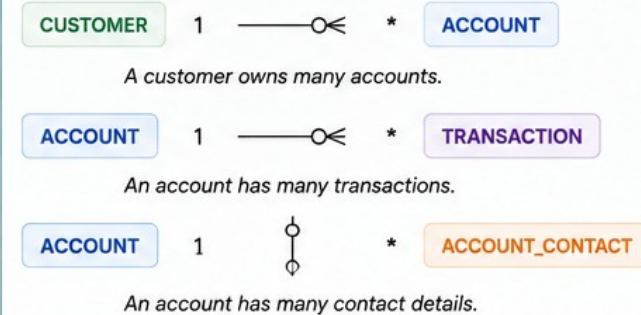
CONTEXT Example: Account Service
Manages customer accounts and related information.

DESIGN PRINCIPLES

- ✓ Data is owned and managed within the service
- ✓ Model based on domain needs
- ✓ Encapsulate data and relationships
- ✓ Use proper keys and constraints
- ✓ Keep model cohesive and consistent



RELATIONSHIP SUMMARY



BENEFITS OF GOOD ENTITY DESIGN



Clear domain modeling



Data integrity and consistency



Easier maintenance and evolution



Better performance and scalability



Encapsulated data within the service

Entity Relationships and Fetch Strategies

Day 3 • Module 12 • Topic 262

Modeling associations between entities and choosing how related data is loaded.



Why It Matters



Relationship mapping
represents real-world
associations in the domain.



Fetch strategies
control when related
data is loaded.

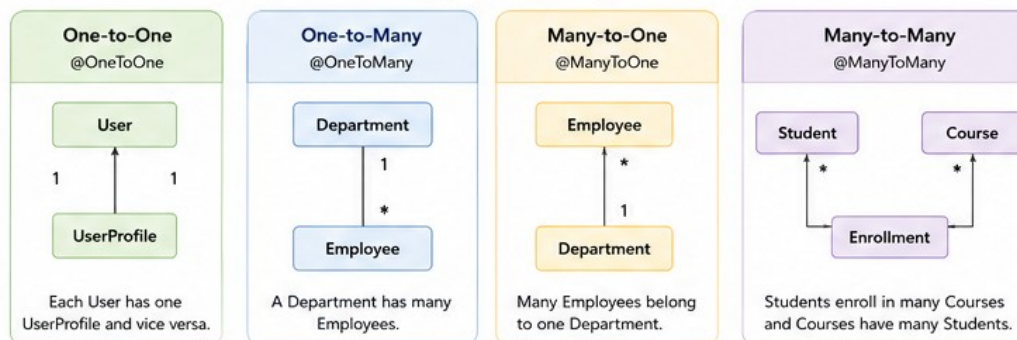


Right choices
improve performance
and avoid common
pitfalls like N+1.

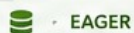


Balance
convenience, performance,
and memory usage.

Common Entity Relationships

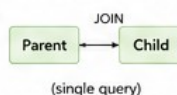


Fetch Strategies



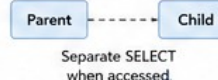
EAGER

- Related entity is loaded immediately in the same query (when possible).
- Use only when the association is always needed.



LAZY (default for @OneToMany, @ManyToOne)

- Related entity is loaded only when accessed for the first time.
- Improves performance when data is not always needed.



Example (JPA)

```
@Entity
class Department {
    @Id @GeneratedValue
    private Long id;

    private String name;

    @OneToMany(mappedBy = "department",
        fetch = FetchType.LAZY,
        cascade = CascadeType.ALL,
        orphanRemoval = true)
    private List<Employee> employees = new ArrayList<>();
}

@Entity
class Employee {
    @Id @GeneratedValue
    private Long id;

    private String name;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;
}
```

Common Fetch Pitfalls



N+1 Select Problem Accessing LAZY collections in a loop issues 1 query for parents + N queries for children.



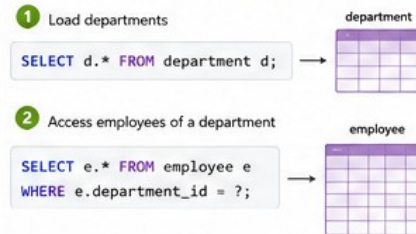
Overusing EAGER Can load large graphs you don't need, increasing response time and memory usage.



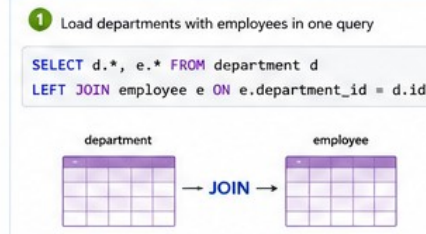
Wrong Fetch with Pagination JOIN FETCH with pagination on collections can return incorrect results or performance issues.

How Fetching Works (Example)

LAZY Loading



EAGER / JOIN FETCH



Fetch Strategy Summary

Strategy	When Data Is Loaded	Pros	Cons	Typical Use
EAGER	Immediately with parent	No extra queries; convenient	Can load unnecessary data; heavy joins	To-One when always needed
LAZY	On first access	Better performance; loads only when needed	Can cause N+1 selects; LazyInitializationException	To-Many, optional To-One
JOIN FETCH	In same query using JOIN	Avoids N+1; single round-trip	Duplicate rows; pagination caveats	Read use cases with known query
EntityGraph / DTO	Customizable per use case	Fetch only required fields/relations	More configuration; separate queries possible	APIs and complex read scenarios

Best Practices

- ✓ Use LAZY as default; override only when needed.
- ✓ Use JOIN FETCH in queries when you know you need the data.
- ✓ Use @EntityGraph or DTO projections to fetch exactly what you need.
- ✓ Avoid accessing LAZY collections in a loop.
- ✓ Test and measure: choose the strategy that fits your use case.



Key Takeaway

Model relationships to reflect your domain, and choose fetch strategies that deliver the right data at the right time—no more, no less.

REPOSITORY ABSTRACTIONS

CrudRepository, PagingAndSortingRepository, and JpaRepository each add a layer of out-of-the-box CRUD, paging, and JPA-specific capability.



WHY USE REPOSITORY ABSTRACTIONS

- Decouples business logic from data access implementation
- Consistent programming model across different data stores
- Easier testing with mock repositories; supports automation of common CRUD operations

KEY TAKEAWAY CrudRepository, PagingAndSortingRepository, and JpaRepository each add a layer of out-of-the-box CRUD, paging, and JPA-specific capability.

LAZY LOADING AND THE N+1 PROBLEM · QUERY METHODS AND CUSTOM QUERIES



THE PROBLEM

- 1 query to load N parents + N queries (one per parent's lazy collection) = N + 1 queries
- Common in REST APIs returning lists with nested data



HOW TO AVOID IT

- **JOIN FETCH:** fetch associations in the same query
- @EntityGraph, DTO projections, and batch fetching each solve it a different way
- Always profile real queries (Hibernate SQL logs, p6spy, Datasource proxy)



FOUR QUERY APPROACHES

- **Derived query methods:** Spring Data parses the method name and builds the query automatically
- **@Query (custom JPQL):** for when the method name would be too long or unreadable
- **Native SQL queries:** for database-specific features or complex queries
- **Named queries:** reusable queries defined with @NamedQuery

KEY TAKEAWAY Loading N parent records and then accessing a lazy collection on each one issues 1 + N queries -- and performance degrades as N grows. • findByEmail, findByStatus, and findByNameContaining are parsed straight from the method name -- no query code required.

DATABASE CONNECTION POOLING · SCHEMA MIGRATION WITH FLYWAY OR LIQUIBASE



WHY POOLING MATTERS

- Establishing a DB connection is expensive (network handshake, authentication)
- Pooling reuses connections, reducing latency and improving throughput
- Prevents exhausting database connections and improves scalability under load



FLYWAY VS. LIQUIBASE

- **Flyway:** SQL-first, simple learning curve, manual (SQL) rollback
- **Liquibase:** SQL, XML, YAML, or Java changesets; more extensible, built-in rollback support



KEY HIKARICP SETTINGS

- maximum-pool-size, minimum-idle, connection-timeout, idle-timeout, max-lifetime
- Right-size the pool to match database capacity and workload

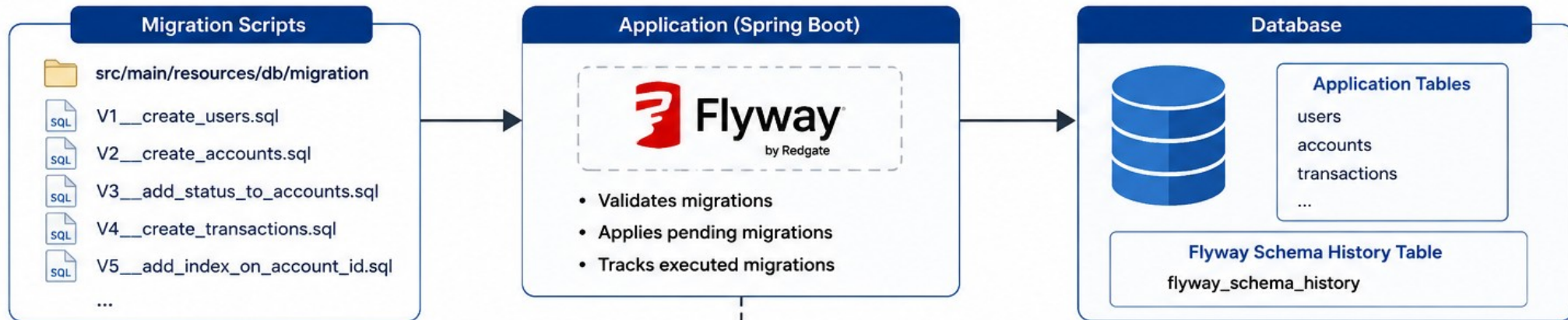
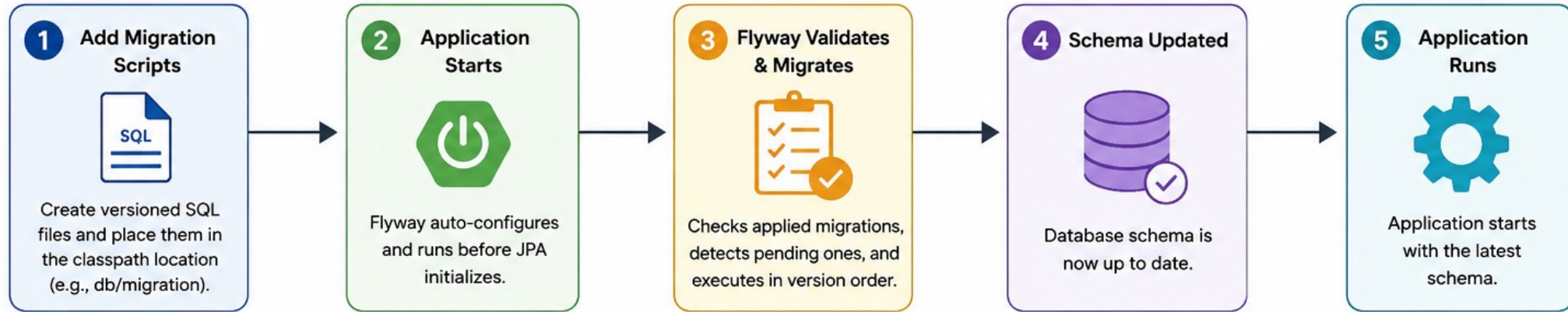


BEST PRACTICES

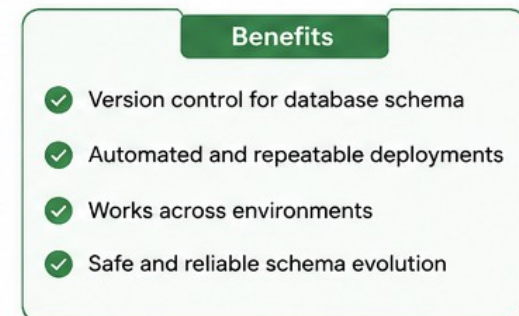
- Always version control migration files; make migrations small, atomic, and focused
- Never edit a migration that has already been applied

KEY TAKEAWAY HikariCP, Spring Boot's default pool, cuts latency and prevents connection exhaustion by reusing a right-sized pool of connections. • Flyway and Liquibase both track applied migrations in a history table so every environment ends up on the exact same schema version.

Flyway Migrations



flyway_schema_history (Tracks Applied Migrations)									
installed_rank	version	description	type	script	checksum	installed_by	installed_on	execution_time	success
1	1	create_users	SQL	V1__create_users.sql	187654321	app_user	2025-05-20 10:15:30	120	true
2	2	create_accounts	SQL	V2__create_accounts.sql	187654322	app_user	2025-05-20 10:15:31	98	true
3	3	add_status_to_accounts	SQL	V3__add_status_to_accounts.sql	187654323	app_user	2025-05-20 10:15:32	75	true
...	...	...	...	...	...	...	...	...	...



DATABASE CONFIGURATION ACROSS ENVIRONMENTS · LOCAL TRANSACTIONS WITH SPRING



TYPICAL ENVIRONMENTS

- **Development:** local or shared DB for developer productivity
- **Test/QA:** isolated DB for testing and automation
- **Staging/UAT:** production-like for final validation
- **Production:** highly available and secured



HOW A LOCAL TRANSACTION WORKS

- Service layer begins a transaction, executes operations, then commits or rolls back
- Default propagation is REQUIRED; runtime exceptions trigger rollback by default
- Managed by Spring's PlatformTransactionManager, applied via @Transactional



BEST PRACTICES

- Never commit passwords or URLs; use environment variables or a secrets manager
- Use the least-privilege DB user and validate configuration at startup

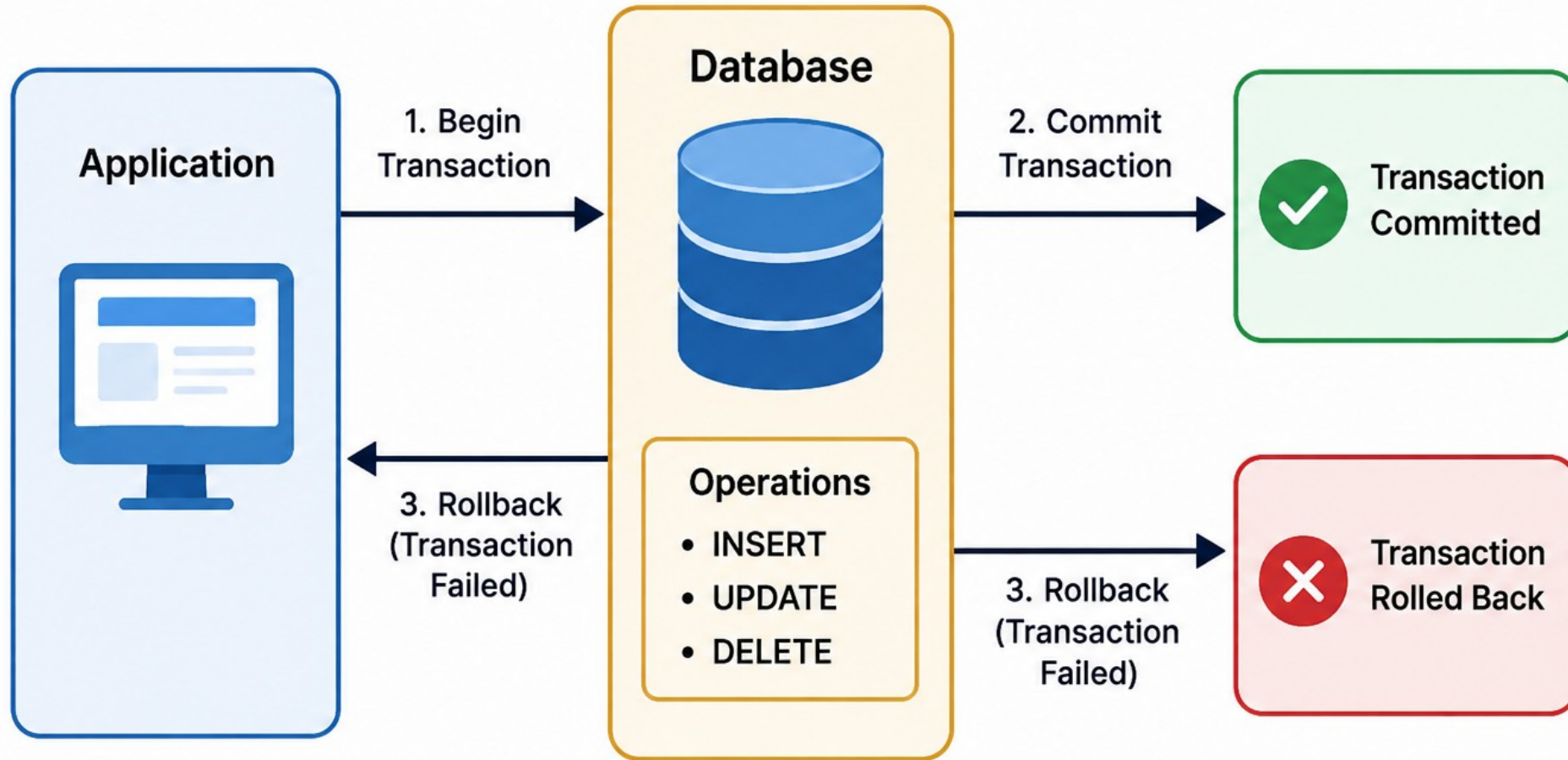


BEST PRACTICES

- Keep transactions short and focused; use @Transactional at the service layer
- Do not perform remote calls (REST, messaging) inside a transaction

KEY TAKEAWAY application-dev.yml, application-qa.yml, and application-prod.yml each point at a different database with different pool sizing and logging. • If any operation inside a @Transactional service method throws, Spring rolls back every change made earlier in that same method.

Local Transactions



TRANSACTION PROPAGATION · TRANSACTION ISOLATION LEVELS

THE FOUR SQL ISOLATION LEVELS

LEVEL	PREVENTS	CONCURRENCY
Read Uncommitted	Nothing	Highest
Read Committed	Dirty reads	High (most systems' default)
Repeatable Read	Dirty + non-repeatable reads	Medium
Serializable	Dirty + non-repeatable + phantom reads	Lowest

KEY PROPAGATION TYPES

- **REQUIRED (default):** join the existing transaction, or create a new one
- **REQUIRES_NEW:** suspend any existing transaction and start an independent one
- **MANDATORY:** must run inside an existing transaction, or throw an exception
- **NEVER:** must not run inside a transaction at all

KEY TAKEAWAY REQUIRED is the sensible default for most business operations; REQUIRES_NEW isolates audit logging or notifications from the main transaction. • READ COMMITTED is the default for most systems and balances consistency against concurrency; SERIALIZABLE trades concurrency for full correctness.

OPTIMISTIC AND PESSIMISTIC LOCKING · WHY DISTRIBUTED TRANSACTIONS ARE DIFFICULT

OPTIMISTIC LOCKING

- Assume no conflicts; validate the version at commit time
- @Version field on the entity is checked and incremented automatically by JPA
- High concurrency, but requires retry logic on conflict

WHY IT'S HARD

- No central coordinator knows the status of every participant at all times
- Network latency, partitions, and inevitable service failures make atomicity unreliable
- Locking across services hurts availability, performance, and scalability

PESSIMISTIC LOCKING

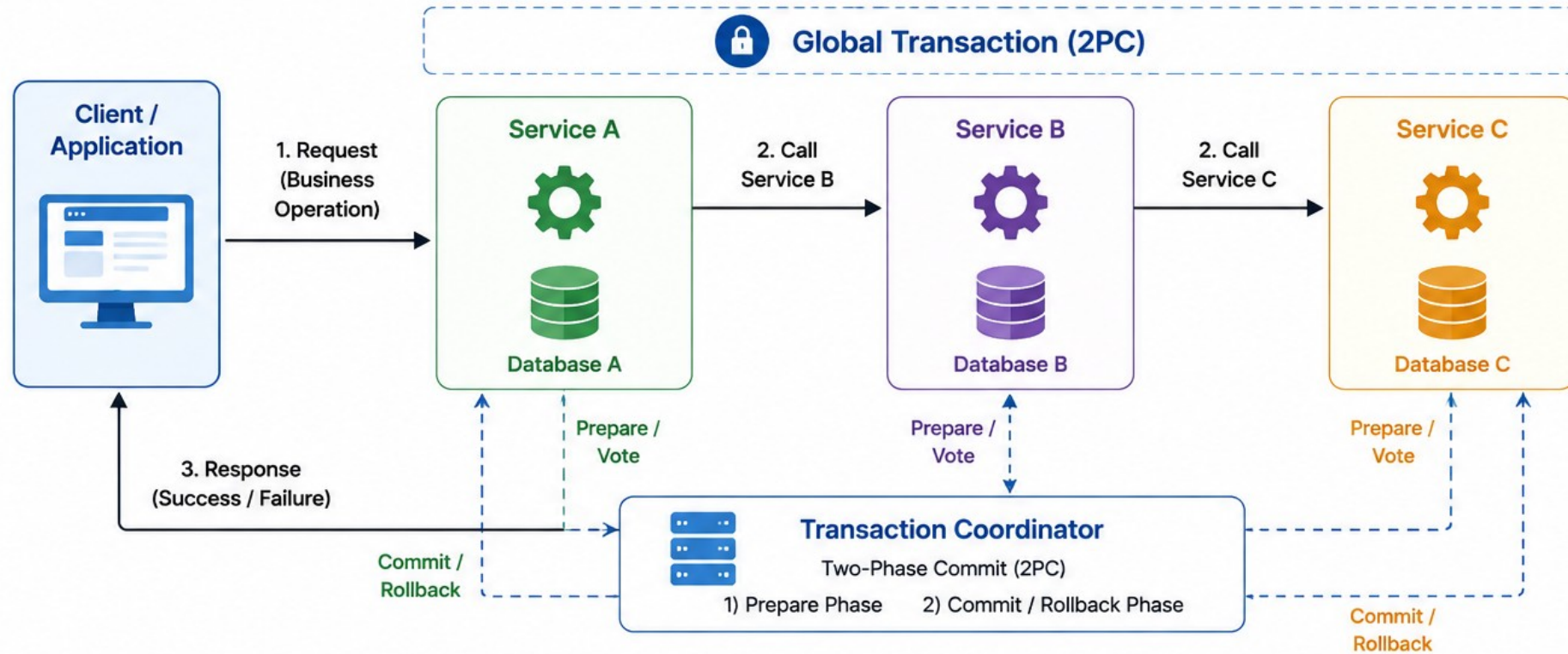
- Assume conflicts; lock the row upfront with @Lock(LockModeType.PESSIMISTIC_WRITE)
- Prevents conflicts outright, but reduces concurrency and risks deadlocks

WHAT MICROSERVICES DO INSTEAD

- Avoid distributed transactions where possible
- Use patterns like Saga, Outbox, and eventual consistency; design for compensation, not rollback

KEY TAKEAWAY A @Version field enables optimistic locking for read-heavy, low-conflict data; PESSIMISTIC_WRITE locks suit high-conflict, write-heavy updates. • Once a service commits, it cannot be rolled back automatically if another downstream service fails -- data is left partially updated.

Distributed Transactions - Why Avoid Them



Why Avoid Distributed Transactions



Performance Overhead

Multiple network round trips (prepare & commit) increase latency and reduce throughput.



Blocking & Locks

Resources are locked across services until the transaction completes.



Single Point of Failure

The coordinator is a critical component whose failure can block the entire transaction.



Poor Scalability

Does not scale well with more services, participants, and higher load.



Reduced Availability

If any participant is slow or down, the whole transaction can fail or time out.



Operational Complexity

Hard to implement, test, monitor, and debug across multiple services.

Better Alternatives



Local Transactions

Keep ACID within each service.



Event-Driven Architecture

Use events to coordinate between services.



Sagas (Choreography / Orchestration)

Break long transactions into a sequence of local transactions with compensation.



Idempotency & Retries

Ensure reliable processing without distributed locks.



Outbox Pattern

Publish events reliably from local transactions.

TWO-PHASE COMMIT AND ITS LIMITATIONS · EVENTUAL CONSISTENCY · SAGA PATTERN OVERVIEW



HOW 2PC WORKS

- **Phase 1 (Prepare):** coordinator asks all participants if they are ready to commit
- **Phase 2 (Commit/Rollback):** if all voted yes, coordinator tells everyone to commit; otherwise, rollback



LIMITATIONS

- **Blocking protocol:** a coordinator failure after Phase 1 leaves participants stuck
- High latency, poor scalability, and not resilient to network partitions



HOW IT WORKS

- A write goes to the primary; the update propagates asynchronously to replicas
- For a window of time, different reads may return different values
- After propagation completes, all nodes converge to the same value



KEY CHARACTERISTICS

- No immediate consistency, but guaranteed convergence and tolerance for failures
- **Trade-off:** sacrifices immediate consistency for availability and scalability



HOW A SAGA WORKS

- Each service completes its own local transaction, then triggers the next step
- If a step fails, previously completed steps are compensated in reverse order
- Can be implemented via orchestration (central coordinator) or choreography (event-driven)



BENEFITS

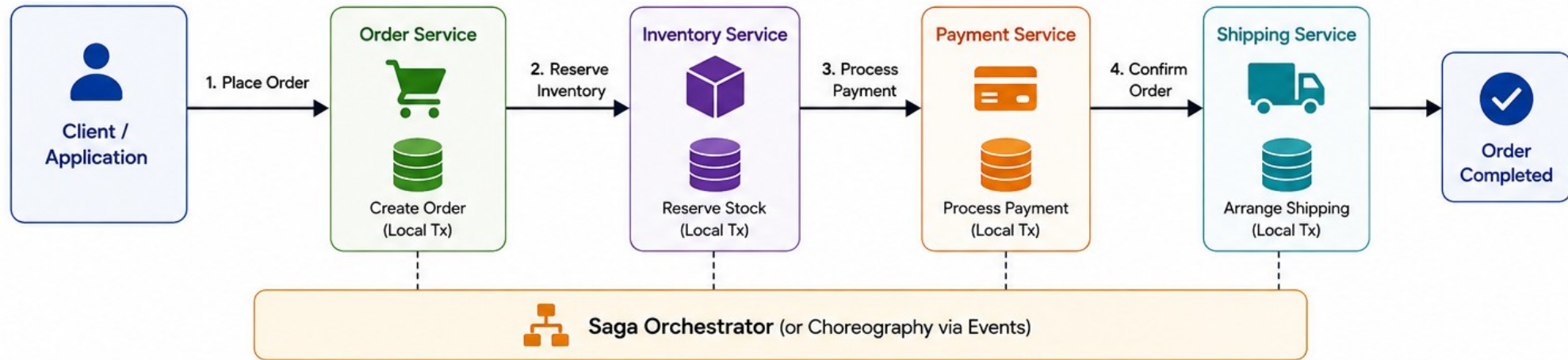
- No distributed locks or 2PC; better scalability, availability, and fault isolation

KEY TAKEAWAY If the coordinator fails after the prepare phase, every participant blocks indefinitely waiting for a decision that may never come.

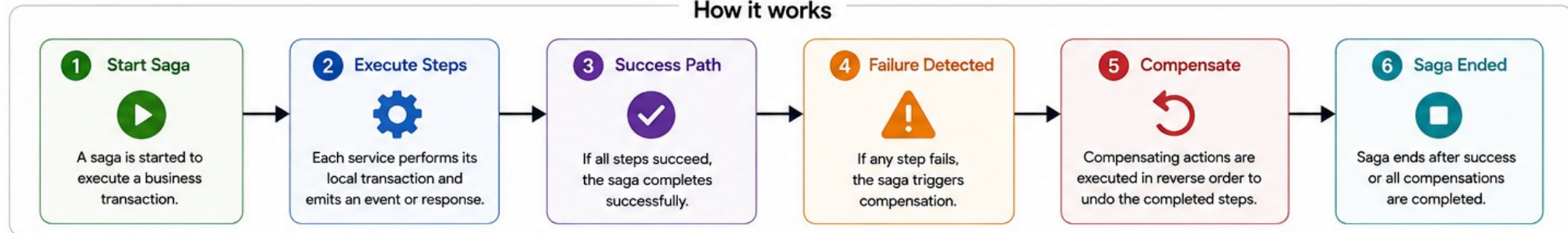
- Different nodes may briefly disagree after a write, but the system guarantees they converge to the same state once updates stop arriving.
- If step three of a saga fails, steps one and two are undone in reverse order through their own compensating actions -- not a database rollback.

Saga Pattern Overview

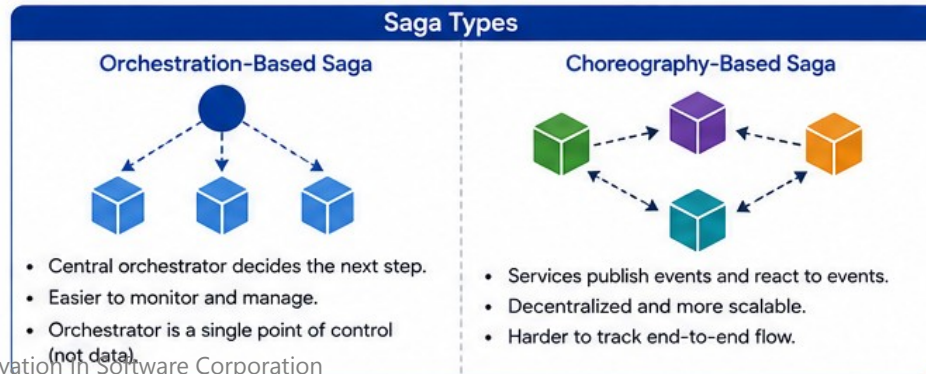
Manage distributed transactions as a sequence of local transactions with compensating actions.



How it works



Saga Types



Key Characteristics

Local Transactions	Each step updates its own database.
Event-Driven	Communication via events or messages.
Eventual Consistency	System reaches a consistent state over time.
Compensating Actions	Used to undo previously completed steps on failure.
No Locks	Avoids long-running locks across services.

CHOREOGRAPHY-BASED SAGAS · ORCHESTRATION-BASED SAGAS · COMPENSATING TRANSACTIONS



HOW CHOREOGRAPHY WORKS

- A service performs a local transaction, then publishes an event describing what happened
- Other services listen for that event and perform their own local transaction and next event



BENEFITS AND CHALLENGES

- Decentralized, highly scalable, and loosely coupled -- no single point of failure
- Harder to visualize the full business flow and to trace/debug across many services



HOW ORCHESTRATION WORKS

- Client requests a business operation; orchestrator starts a saga instance
- Orchestrator sends a command to each participant, records progress, and proceeds
- On failure, the orchestrator triggers compensating transactions in reverse order



ADVANTAGES AND CONSIDERATIONS

- Centralized control, visibility, and easier error handling
- The orchestrator becomes a critical component and a potential bottleneck



EXAMPLES OF COMPENSATION

- Cancel an order, release reserved inventory, refund a payment
- Executed only when a failure occurs, always in reverse order of the completed steps



CONSIDERATIONS

- Compensation actions must be carefully designed and idempotent
- Not always possible to perfectly undo an action -- the business must tolerate temporary inconsistency

KEY TAKEAWAY TransactionSubmitted is consumed by Transaction Service and later by Risk Assessment -- all without a shared database. • AccountOpeningOrchestrator sends commands to Customer, Account, and Notification services and triggers compensations if any step fails. • Closing an account that was opened but never funded, or reversing a transaction, brings the system to an acceptable state, not a rewritten history.

DATA CONSISTENCY TRADEOFFS · CQRS PATTERN OVERVIEW

✚ CONSISTENCY REQUIREMENTS BY OPERATION

OPERATION	CONSISTENCY NEED
Deduct funds from account	Strong consistency inside the owning service
Send notification	Eventual consistency acceptable
Update reporting dashboard	Eventual consistency acceptable
Validate login	Immediate response required

📌 COMMAND VS. QUERY RESPONSIBILITY SEGREGATION

- **Write side:** commands validate business rules and update the source-of-truth data model
- **Read side:** queries run against a denormalized, purpose-built read model
- The read model is updated asynchronously, typically by consuming domain events

✅ WHEN CQRS HELPS

- Reporting and dashboard use cases that would otherwise require expensive cross-service joins
- Read and write workloads that need to scale independently

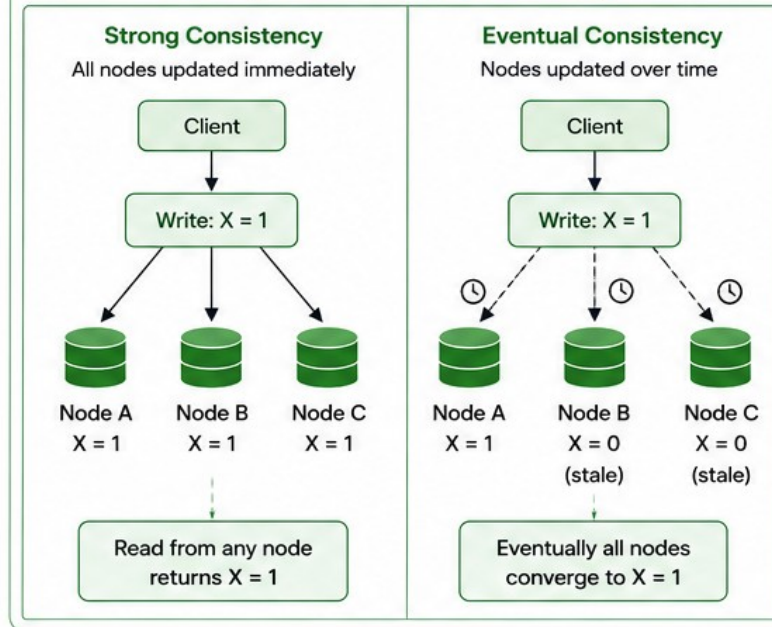
KEY TAKEAWAY Deducting funds needs strong consistency; updating a reporting dashboard can tolerate eventual consistency -- the business decides which. • Commands update the owning service's write model; queries run against a purpose-built read model kept in sync through events.

Consistency Models and Eventual Consistency

Consistency Models

Model	Description
 Strong Consistency	Every read returns the latest write. All nodes see the same data at the same time.
 Linearizability	Operations appear to take effect atomically at some instant in time between invocation and response.
 Eventual Consistency	If no new updates are made, all replicas will eventually converge to the same data.
 Causal Consistency	Causally related writes are seen in the same order by all nodes.
 Session Consistency	Reads within a session see writes from that session.

Strong vs Eventual Consistency



When to Use Which?

Use Strong Consistency When:



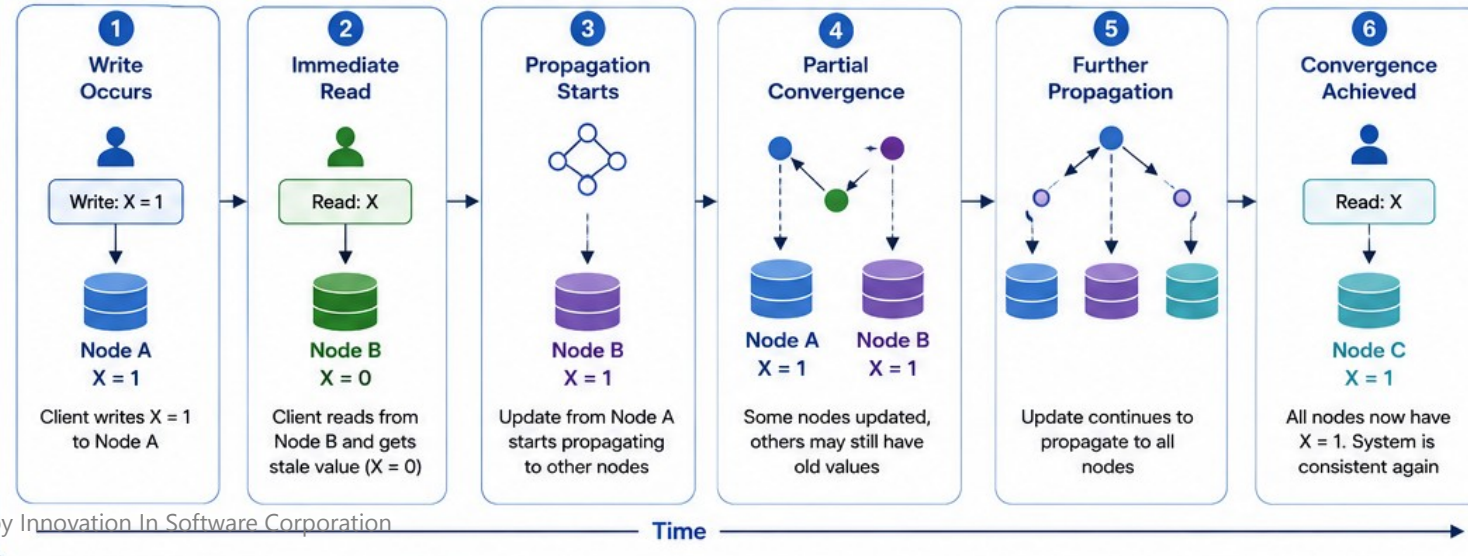
- Financial transactions (banking, payments)
- Inventory with low tolerance for oversell
- Security / access control decisions
- Critical configuration changes

Use Eventual Consistency When:



- Social feeds, likes, comments
- Product catalogs, search indexes
- Analytics, logging, monitoring
- High availability and partition tolerance are priorities

Eventual Consistency in Action



Key Characteristics of Eventual Consistency



Not Immediate
Updates may not be visible across all nodes right away.



Converges Over Time
If no new updates occur, all nodes will reach the same state.



Partition Tolerant
System remains available even when network partitions occur.



High Availability
Prioritizes availability and scalability over immediate consistency.



Use with Idempotency
Consumers should be idempotent to handle duplicates or reordering.

API COMPOSITION FOR DISTRIBUTED DATA · DATA PRIVACY AND SERVICE BOUNDARIES · DATABASE PERFORMANCE CONSIDERATIONS



HOW IT WORKS

- A composition layer calls each owning service for its slice of the data
- Results are merged in application code to build one client-facing response
- Useful for read-heavy views that combine data owned by multiple services



COMMON PERFORMANCE ISSUES

- Missing indexes, inefficient queries, and N+1 query problems
- Loading too much data, large response payloads, poor connection pool settings



PRIVACY-AWARE DATA DESIGN

- The owning service controls exactly what fields it exposes through its API
- Downstream services store only the identifiers they need, not full customer records
- Events should avoid carrying sensitive fields unless explicitly required and protected



BEST PRACTICES

- Add indexes for common lookup fields (customer email, account number, transaction date)
- Measure before optimizing; use pagination and DTO projections for large or summary result sets

KEY TAKEAWAY A composer calls Customer, Account, and Transaction services in parallel and joins their responses in application code, not in SQL. • No Customer Service in this capstone: Account stores customerId as a field; other services keep only accountId and request data through APIs. • Missing indexes and N+1 queries are the most common root causes of a slow banking API -- and both are fixable before caching is even considered.

DATA PERSISTENCE ANTI-PATTERNS · CACHING FUNDAMENTALS · WHEN CACHING HELPS-AND WHEN IT DOES NOT

ANTI-PATTERNS TO AVOID

- Returning JPA entities directly from APIs instead of DTOs
- **Sharing a database across services; using ddl-auto:** update in production
- Ignoring lazy loading behavior and eager-loading everywhere
- Creating extremely large, deeply nested entity graphs

GOOD CANDIDATES

- Frequently read data that is expensive to compute or changes infrequently
- Reference data and aggregated read models

WHAT A CACHE STRATEGY MUST DEFINE

- What data is cached and where it is cached (local vs. distributed)
- How long it lives (TTL) and when it is invalidated
- What happens to the application when the cache is unavailable

POOR CANDIDATES

- Highly sensitive data, rapidly changing balances
- Data requiring immediate accuracy, large unbounded result sets

KEY TAKEAWAY Returning JPA entities directly, sharing databases, and skipping migrations are small shortcuts with outsized long-term cost. • What to cache, where, for how long, and how to invalidate it are all design decisions, not defaults to accept blindly. • Account balances and fraud scores are poor caching candidates; reference data and product configuration are excellent ones.

CACHE-ASIDE PATTERN · READ-THROUGH AND WRITE-THROUGH CACHING · WRITE-BEHIND CACHING

CACHE-ASIDE FLOW

1. Application checks the cache for the requested key
2. If found, return the cached value directly
3. If not found, query the database, store the result in the cache, then return it

WRITE-THROUGH

- Writes go to the cache first, which synchronously writes through to the database
- Keeps cache and database aligned, at the cost of added write latency

READ-THROUGH

- On a cache miss, the cache itself loads the value from the database and stores it
- Application code always talks to the cache, never directly to the database for reads

HOW WRITE-BEHIND WORKS

- Write goes to the cache, which returns immediately
- The cache asynchronously flushes the write to the database, often batched
- Improves write latency and throughput, but risks data loss if the cache fails before flushing

KEY TAKEAWAY @Cacheable implements cache-aside out of the box -- the application, not the cache, owns the responsibility of keeping data fresh. • Instead of the app deciding when to populate the cache, the caching layer itself loads on a miss and writes through on every update. • Fast writes come at the cost of a short window where cache and database can disagree if the service fails before the write flushes.

REDIS ARCHITECTURE AND DATA STRUCTURES · SPRING CACHE ABSTRACTION · INTEGRATING REDIS WITH SPRING BOOT

REDIS DATA STRUCTURES

- **Strings and hashes:** common for basic caching of values and objects
- **Lists, sets, and sorted sets:** queues, uniqueness checks, and leaderboards
- **Streams:** lightweight event/message logs

TYPICAL SETUP

- `spring.data.redis.host` / port configuration in `application.yml`
- `@EnableCaching` on the application class activates Spring's cache abstraction
- `@Cacheable(value = "customers", key = "#id")` caches a repository lookup method

CORE CACHING ANNOTATIONS

- **@Cacheable:** cache the method's return value, keyed by its arguments
- **@CachePut:** always run the method and update the cache with the result
- **@CacheEvict:** remove an entry (or clear a cache) when data changes

IMPORTANT CAUTION

- Redis should not casually become the system of record for critical banking data
- The relational database remains the source of truth; Redis acts as a cache

KEY TAKEAWAY Strings, hashes, lists, sets, sorted sets, and streams give Redis far more flexibility than a plain cache -- rate limiting, queues, and counters included. • `@Cacheable`, `@CachePut`, and `@CacheEvict` express caching intent declaratively -- the actual cache backend is a configuration decision. • Lecture only — Labs 1-5 do not install Redis (PostgreSQL is the store). `spring.data.redis.host/port` plus `@Cacheable` is enough to cache a lookup method.

CACHE KEYS AND VALUE DESIGN · CACHE EXPIRATION AND TIME-TO-LIVE · CACHE INVALIDATION STRATEGIES



CACHE KEY DESIGN PRINCIPLES

- Namespace keys by service and entity type, e.g. customer:101:profile
- Keep values small and focused -- cache DTOs or summaries, not full entity graphs
- Use consistent, meaningful key patterns so keys can be found, scanned, or bulk-invalidated



MATCHING TTL TO DATA

- **Account status:** short TTL
- **Customer profile:** moderate TTL
- **Product configuration:** longer TTL
- **Fraud risk score:** very short TTL



INVALIDATION TRIGGERS

- Time-to-live expired
- Underlying data was updated (evict via @CacheEvict on the write path)
- Memory limit reached, or manual invalidation was triggered

KEY TAKEAWAY customer:101:profile beats a bare 101 -- namespacing prevents key collisions and makes bulk invalidation by prefix possible. • Account status needs a short TTL, fraud risk score needs a very short TTL, and product configuration can tolerate a much longer one. • When a customer updates their phone number, the write must update the database and evict or refresh the cached profile in the same operation.

PREVENTING STALE DATA · CACHE STAMPEDE AND PENETRATION



PREVENTING STALE DATA

- Evict or update the cache in the same transaction or operation as the database write
- Use short TTLs for data where staleness has real business consequences
- Never cache highly volatile data (account balances) without an explicit invalidation path



TWO RELATED FAILURE MODES

- **Cache stampede:** a hot key expires and many concurrent requests all recompute it at once
- **Cache penetration:** repeated requests for a nonexistent key never get cached, always hitting the database



MITIGATIONS

- Locking or request coalescing on cache-miss recomputation
- Cache negative results briefly, or use a bloom filter to short-circuit lookups for known-missing keys

KEY TAKEAWAY Pairing every cache-populating read with a cache-evicting write is the single most important discipline in cache design. • Cache penetration is the related risk of repeated lookups for keys that don't exist at all, bypassing the cache on every single request.

DISTRIBUTED CACHING CONSIDERATIONS · REDIS HIGH AVAILABILITY · SECURING REDIS



KEY CONSIDERATIONS

- Partitioning/sharding strategy across the cluster as data volume grows
- Consistent key design across every service instance sharing the cache
- Defined fallback behavior for the application when the cache cluster is unavailable



HIGH AVAILABILITY OPTIONS

- Primary-replica replication for read scaling and failover
- Redis Sentinel for automatic failover and monitoring
- Redis Cluster for sharded, horizontally scaled deployments



SECURING REDIS

- Require authentication (requirepass or Redis ACLs) -- never run open
- Restrict network access to only the services that need it
- Enable TLS for data in transit, and avoid caching sensitive data unless explicitly protected

KEY TAKEAWAY Multiple service instances sharing one Redis cluster must agree on key design, eviction policy, and what happens if the cluster is unreachable.

- A single Redis instance is a single point of failure -- replication with automatic failover is what makes caching production-safe.
- Authentication, network isolation, and TLS are not optional extras for Redis in a banking platform -- they're baseline requirements.

Monitoring Cache Performance

Measure. Observe. Optimize.



Effective monitoring ensures your cache delivers **speed**, **reliability**, and **cost efficiency**.

WHY MONITOR CACHE?



Ensure Performance
Track hit rates and latency to ensure expected speed.



Ensure Reliability
Detect issues early and prevent outages.



Optimize Cost
Right-size memory and avoid over-provisioning.



Capacity Planning
Understand usage trends to plan for growth.

BEST PRACTICES

- ✓ Define and monitor SLOs for cache metrics.
- ✓ Alert on anomalies and threshold breaches.
- ✓ Correlate cache metrics with application performance.
- ✓ Review eviction causes and memory fragmentation.
- ✓ Regularly analyze key patterns and TTL usage.

KEY CACHE METRICS

Hit Rate



% of requests served from cache. Higher is better.

Target: > 80%

Miss Rate



% of requests not in cache.

Target: < 20%

Latency



Time to get a response from cache (p50, p95, p99).

Target: Low & Stable

Evictions



Keys removed due to memory pressure.

Target: Low

Memory Usage



Used memory vs. total allocated memory.

Target: < 85%

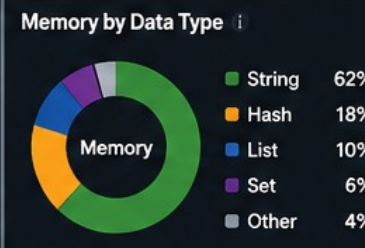
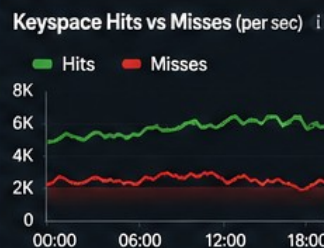
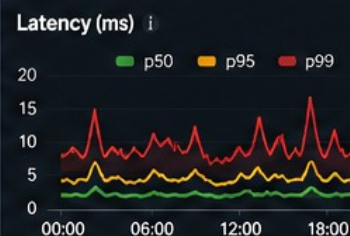
Throughput



Commands per second (reads + writes).

Monitor Trends

DASHBOARD EXAMPLE (Redis + Prometheus + Grafana)



Remember:

A cache is fast only when it hits, healthy when it's stable, and valuable when it's right-sized.



Tools:

Redis INFO, Prometheus, Grafana, Alertmanager, OpenTelemetry, and Application Metrics



EXERCISE 2.1 — MAP DATA OWNERSHIP FOR ACCOUNT VS TRANSACTION

Module 4 checkpoint A — Separate DBs/schemas; no cross-service tables.



OBJECTIVE

- Separate DBs/schemas; no cross-service tables
- 10 min · design checkpoint
- **Worksheet:**
curriculum/day-02/exercises/exercise-2.1-data-ownership.md



DO THIS

1. Account table lives only in account_db. Transaction table lives only in transaction_db
2. Transaction stores accountId as a reference, not a foreign key across services
3. Eligibility is a REST GET to Account Service, not a SQL join
4. Redis is lecture-only — not part of this capstone



EXPECTED RESULT

- A two-column ownership map with zero shared tables.
- Lab 2 uses transaction_db on host port 5434. Never join to account_db.

KEY TAKEAWAY A two-column ownership map with zero shared tables.